

by the user

```
session.use_strict_mode=0n
```

```
; Enforce HTTPS (SSL/TLS), if the site supports it
```

```
session.cookie_secure=0n
```

Many settings can also be updated using `ini_set()`. For example:

```
ini_set('session.cookie_httponly', '1');
```

In this case, make sure to place these calls before `session_start()`, preferably at the very beginning.

To get authentic session security advice, visit <https://www.php.net/manual/en/session.security.php>.

Session conflict

The values you get from `$_SESSION` are kept on disk in places like `/var/lib/php/sessions/`. Needless to say, it should have tight permissions. But even with this place being protected, attacks can be possible. Think of a server in which two different sites share the same PHP installation/environment. Both sites use sessions, and the session keys of both are stored in the cookie `PHPSESSID` on the client-side.

No problem — since the browser will check the domain name before sending back the cookie. But what if the visitor intentionally sends the cookie to the wrong site? If Site A sets the cookie and the user sends it to Site B, the latter will accidentally read the session data from Site A because they share the storage area. If the user has logged in to Site A with a user name with which there exists a user in Site B, he might actually be permitted to use the site as that user.

Setting special keys in `$_SESSION` won't help since they can be scanned. The best option is to use a custom storage path and safeguard it with permissions. To set a custom path, update `session.save_path`, either in `php.ini` or via `ini_set()`.

Equality checks

The ability of PHP functions to return values of any type may feel like a blessing. On success the function can return the result and on failure, the function can return `FALSE`. There's no need for exceptions or additional error flags. Most of the built-in functions follow this pattern.

Although convenient, this can be a dangerous feature if your error checking is faulty.

`==` and `!=` will confuse `FALSE` with `0` and `TRUE` with any non-zero values. This is not a problem in many other languages since functions cannot return values of multiple types. But in PHP, if your `_get_user_id()` returns `FALSE` on failure and an integer otherwise, the call will mistakenly be understood to have failed because it returned a valid user ID `0`; and comparing it with `FALSE`, using `==`, finds a match.

Read the documentation before using any function and see if it has this pitfall. Use `===` and `!==` (which are type-sensitive) instead of `==` and `!=` for error checking if the function returns something like `TRUE`, `FALSE` or `NULL`. Although we stressed on return values, these operators can be used on regular variables also.

Where to place the scripts

All the user-accessible files are put inside the Document Root (usually called `public_html`) and its children. This includes PHP scripts, which are executed instead of being displayed. But when something goes wrong (bad configuration or a bug in your server software), scripts may get displayed in the source form. This shouldn't cause any security issues since your site has to be designed in a way that it is secure even if its full source is revealed to

the world. But what about things that should be kept secret? Like the MySQL password (which cannot be hashed)? Or some experimental code?

You can place them somewhere outside the document root, and include them in user-accessible files using `include()` or `require()`.

Own server versus shared hosting

If Website A is the only one hosted on the server, all you have to worry about is how to protect your site and data from visitors (not that this is easy). But if the site is on a shared server, you have to worry about other sites accessing and interfering with the data and operations of the site in question. Even if all other sites on the server are owned and operated by you, problems can arise. We've discussed one case already regarding session conflict and how to mitigate it using `session.save_path`.

File permissions

Regardless of where they are placed, any of your files can be read (and even modified) by other sites in a shared hosting environment, if you didn't set the right permissions. While setting permissions, 'others' means other users/sites on the server, not your visitors. So feel free to disable even read access for others.

Sometimes, setting owner-only permissions might prevent the server software from accessing your files because it runs as a separate user. In an ideal shared-hosting environment, Apache or similar software should run each site as its own user. You can easily accomplish this by installing appropriate modules if you own the server. **END** 

By: Nandakumar Edamana

The author is a developer and technology writer who primarily focuses on software freedom, privacy and other ethical issues. He has authored a couple of books and several articles for mainstream publishers. A GNU/Linux user for more than ten years, he releases his software products under free software licences, which are available on nandakumar.co.in.